Pub/Sub and Fan-Out

one event, many independent subscribers

Watch the 4-min video

In one line: publish/subscribe sends each message to every subscriber, so the producer publishes once and new consumers subscribe without changing it; give each subscriber its own queue or consumer group so a slow one can't hold back the others, and use filtering to keep subscribers to what they need.

1 Who needs to know?

An order is placed. The email service needs to know. So do analytics, inventory and, next month, a new fraud service. If the order service calls each one directly, it grows a new dependency for every consumer, and becomes as slow and fragile as the slowest one (lesson 23). With **publish/subscribe**, it publishes one event to a **topic**, and every **subscriber** gets its own copy.

2 Queues vs pub/sub

In a **queue**, consumers compete, and each message is handled once. In **pub/sub**, every subscription receives every message. Most real systems combine them: pub/sub to fan out to services, and within each service a queue or consumer group so its instances share that service's copy.



Figure 1. Three ways to implement fan-out.

WORKED EXAMPLE: AN SNS TOPIC FANNING OUT TO THREE SQS QUEUES

The order service publishes `OrderPlaced` to one SNS topic. Three SQS queues subscribe: email, analytics and fraud. Each queue has its own workers, retry settings and dead-letter queue. The fraud subscription uses an SNS **filter policy** so it receives only orders above 500. When the analytics workers fall behind during a sale, only the analytics queue grows; emails still go out on time. Adding a loyalty service next month means subscribing a fourth queue, with no change to the order service.

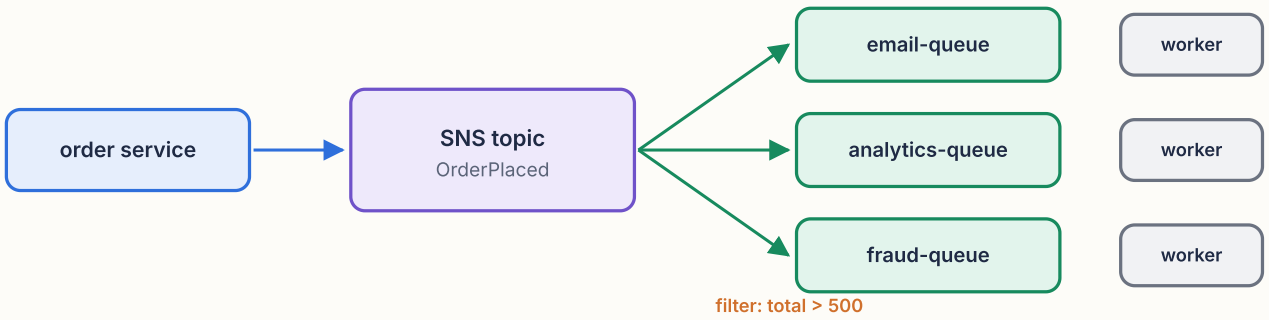


Figure 2. Topic → queue per subscriber: isolation, independent retries and filtering.

3 Things to watch



Figure 3. Pub/sub's common issues.

- Give each subscriber **its own buffer** (queue or consumer group); pushing directly to many HTTP endpoints couples their availability to yours.
- Delivery is usually at-least-once and, across subscribers, unordered: make each subscriber idempotent.
- With many subscribers, the event schema is a public contract (lesson 39).
- Fan-out can multiply load: one event per follower for a celebrity post is millions of writes; use batching or pull-on-read for very large fan-outs.

COMMON MISTAKES

- Multiple services sharing one queue and stealing each other's messages.
- Subscribers sharing one consumer group in Kafka by accident.
- No DLQ per subscription.
- Filtering in every consumer instead of at the subscription.

4 Key terms

Publish/subscribe: each message goes to every subscription

Topic: named channel publishers send to

Subscription: a subscriber's own copy of the stream

Fan-out: one message delivered to many consumers

Filter policy: subscription-side rule selecting messages

5 Interview questions

> **Q1. What's the difference between a queue and pub/sub?** BASIC
> In a queue, consumers compete and each message is processed once; in pub/sub, every subscriber receives its own copy of every message.

Q2. Why put a queue behind each subscription? BASIC

So each subscriber consumes at its own pace with its own retries and DLQ, and a slow or failing one doesn't affect the others or the publisher.

Q3. Design notifications when an order is placed, for email, SMS and analytics. SCENARIO

Publish OrderPlaced to a topic; subscribe one queue per channel with its own workers, retries and DLQ; filter where useful; make each consumer idempotent.

Q4. How do you fan out on Kafka? SCENARIO

Publish once to a topic; each service uses its own consumer group, so each receives every record, and instances inside a group share partitions.

Q5. What's the risk of very large fan-out? TRICKY

Write amplification and load spikes, as with timelines for accounts with millions of followers; mitigate with batching, rate limits, or computing on read for the largest fan-outs.

Q6. Does pub/sub guarantee order across subscribers? TRICKY

No. Each subscription processes at its own pace, so different subscribers can see events at different times, and order within each depends on the broker's per-key guarantees.

Next: Phase 5 · SD-K-001 · Availability Calculations backendroadmap.com